
AdaptMath

Adaptivní vzdělávání pro matematické disciplíny

Technická dokumentace

Task Checker — nástroj pro plnění a kontrolu úloh

Projekt: TQ23000206 (TA ČR)

Hlavní řešitel: Mgr. Jana Medková, Ph.D.

Pracoviště: Fakulta informatiky a managementu, Univerzita Hradec Králové

Doba řešení: 09/2025 – 02/2029

Autor dokumentace: Ing. Dominik Palla, Ph.D.

Verze: 1.1

Datum: 1. června 2026

Obsah

1	Úvod	2
1.1	Cíl projektu	2
1.2	Co řeší tato dokumentace	2
1.3	Tým	2
2	Architektura	2
2.1	Technologický stack	2
2.2	Struktura repozitáře	3
2.3	Tok dat	3
3	Datový model	3
3.1	Tabulka <code>math_tasks</code>	3
3.2	Konvence <code>task_id</code>	4
3.3	Struktura <code>results</code>	4
3.4	Typy výsledků	4
4	Vektor znalostí a kategorie	4
4.1	Dvě úrovně anotace úlohy	4
4.2	KNOWLEDGE_WEIGHTS (61 vah, verze 260601_v2)	5
4.3	TASK_CATEGORIES, TASK_PROPERTIES, TASK_TYPES, TASK_SKILLS	6
4.4	Kompletní seznam vah	6
5	Webové UI: Task Checker	8
5.1	Routes	8
5.2	Přihlášení a session	8
5.3	Seznam úloh (<code>/tasks</code>)	8
5.4	Editor úlohy (<code>/tasks/<id></code>)	9
5.5	Tipy pro vyhodnocení MathLive Compute Engine	10
6	Datový obsah	10
6.1	Zdroj úloh	10
6.2	Pokrytí	10
6.3	Distribuce typů výsledků	10
7	Spuštění lokálně	11
7.1	Závislosti	11
7.2	První spuštění	11
7.3	Konfigurace přes env vars	11
7.4	Opětovné nasazení po změně schématu	11
8	Nasazení do produkce (UHK)	11
8.1	Architektura nasazení	12
8.2	Systemd unit	12
8.3	Provozní příkazy	13
8.4	Známá omezení	13
9	Co dál (roadmap)	13
9.1	Krátkodobé	13
9.2	Střednědobé	13
9.3	Dlouhodobé	13

10 Reference**14**

1. Úvod

1.1. Cíl projektu

AdaptMath je projekt aplikovaného výzkumu TA ČR (TQ23000206, program SIGMA), jehož cílem je navrhnout a pilotně ověřit nové přístupy k adaptivnímu vzdělávání v matematických oborech na vysokých školách. Klíčovým prvkem je vytvoření *adaptivního engine*, který na základě profilu studenta, klasifikace úloh a jeho odpovědí dynamicky přizpůsobuje výběr další úlohy a tvorbu personalizovaného kurzu.

Plánované výstupy projektu:

- **TQ23000206-V2** — adaptivní engine pro matematické kurzy (software v Pythonu),
- **TQ23000206-V3** — adaptivní kurzy pro výuku matematiky (integrace v Moodle).

1.2. Co řeší tato dokumentace

Dokumentace popisuje **Task Checker** — pomocnou webovou aplikaci určenou pro fázi *plnění a kontroly obsahu*. Aplikace umožňuje výzkumnému týmu:

- importovat matematické úlohy ze zdrojových `.tex` skript do databáze,
- vizuálně procházet a ověřovat zadání i jejich očekávané výsledky,
- editovat všechny atributy úlohy (zadání, výsledky, kategorie, vektor vah, IRT parametry),
- testovat odpovědi pomocí **MathLive Compute Engine** stejným způsobem, jakým je bude vyhodnocovat budoucí studentský engine.

Adaptivní engine (IRT, BKT, personalizace) zatím není součástí — bude přidán v následující fázi projektu.

1.3. Tým

Člen	Role	Odpovědnost
dr. Jana Medková	hlavní řešitel	vize datového modelu, vedení projektu
dr. Jiří Haviger	IRT engine	<code>py-irt</code> , kalibrace obtížnosti
dr. Eva Ševčíková	BKT modely	<code>py-bkt</code> , učící parametry α, β
dr. Petr Bauer	obsah	spoluauteur skriptu <i>Základy matematiky 1</i>
dr. Štefan Šteflovíč	člen týmu	
Ing. Dominik Palla, Ph.D.	vývoj	frontend, integrace, LLM funkce

2. Architektura

2.1. Technologický stack

Vrstva	Technologie
Backend	Python 3.10+, Flask 3.1, SQLAlchemy 2.x, psycopg2
Databáze	PostgreSQL 15 (v Dockeru)
Frontend (render)	KaTeX 0.16 — sazba matematiky
Frontend (vstup)	MathLive (<code><math-field></code> web component)
Vyhodnocení odpovědí	MathLive Compute Engine (LaTeX $\rightarrow$ MathJSON)
Šablony	Jinja2 (server-side render)
Kontejnerizace	Docker Compose
Frontend libraries	CDN (jsdelivr, unpkg) — žádný build krok

2.2. Struktura repozitáře

```

adapt_math/
  app.py                # Flask aplikace (routes, API)
  database.py           # init_db (SQLAlchemy engine + session)
  model.py              # ORM model MathTask
  seed_db.py            # drop & recreate + napln. uloh
  docker-compose.yml    # PostgreSQL kontejner
  tasks/
    __init__.py         # def. uloh, jeden soubor = jedno cviceni
    knowledge_weights.py # agreguje ALL_TASKS
    cv01.py ... cv13.py # ulohy ze skripta "Zaklady matematiky 1"
  templates/
    tasks_list.html     # seznam vseh uloh (filtr)
    task_checker.html    # editor jedne ulohy + sandbox

```

2.3. Tok dat

1. Zdrojové .tex skripty (cvičení 1–13) jsou ručně přepsány do strukturovaných Python slovníků v `tasks/cvNN.py` (LLM-asistovaná extrakce, viz sekce 6).
2. `seed_db.py` smaže a znovu vytvoří tabulku `math_tasks` a vloží do ní všech 415 úloh přes SQLAlchemy.
3. `app.py` (Flask) servíruje webové UI; přes endpoint `POST /api/tasks/<id>` expert ukládá změny.
4. Frontend MathLive Compute Engine vyhodnocuje studentské odpovědi symbolicky / numericky, bez backend kola.

3. Datový model

3.1. Tabulka `math_tasks`

Definováno v `model.py` (SQLAlchemy ORM). Aktuálně jediná tabulka.

Sloupec	Typ	Popis
<code>task_id</code>	String (PK)	Identifikátor stylu <code>cvNN_i</code> (např. <code>cv04_5</code>).
<code>content_latex</code>	String	Zadání úlohy v LaTeXu; KaTeX render. Pure-math (bez \$) se v UI automaticky obalí do <code>\$\$...\$\$</code> .
<code>results</code>	JSON	Pole pojmenovaných výsledků (viz 3.3).
<code>category</code>	String	Primární téma úlohy — jeden řetězec z 45 <code>TASK_CATEGORIES</code> (jen SŠ + VŠ matematika, bez vlastností/typu/dovedností).
<code>properties</code>	JSON	Multi-select tagy: list ze 9 <code>TASK_PROPERTIES</code> (Vlastnosti — lineární, kvadratická, mocninná, ...).
<code>task_type</code>	JSON	Multi-select tagy: list z <code>TASK_TYPES</code> (zatím 1 položka — <i>Aplikační</i>).
<code>skills</code>	JSON	Multi-select tagy: list ze 6 <code>TASK_SKILLS</code> (Dovednosti — aplikace vzorce, vytýkání, roznásobení, výpočet rovnic/nerovnic, derivování).
<code>knowledge_vector</code>	JSON	Slovník <code>{name: int 0-100}</code> ; podíl všech 61 skill-komponent pro distribuci přírůstku znalosti. Ukládají se jen non-zero hodnoty.
<code>cognitive_load</code>	String	Klasifikace A–F (typ / praktičnost úlohy).

<code>graph_vector</code>	JSON	<i>Deprecated.</i> Starší tag-vektor (list stringů). Zachováno pro kompatibilitu, v UI se needituje.
<code>irt_difficulty</code>	Float	IRT obtížnost (± 3). Doladí Haviger.
<code>irt_discrimination</code>	Float	IRT diskriminace ($\pm 2,5$).

3.2. Konvence `task_id`

Formát `cvNN_i`, kde:

- `NN` je dvouciferné číslo cvičení (01–13),
- `i` je pořadové číslo úlohy v rámci cvičení (1, 2, ...).

Příklady: `cv01_4` (4. úloha `cv01`), `cv12_58` (poslední úloha `cv12`).

3.3. Struktura `results`

Pole `results` drží **kolekci pojmenovaných výsledků**. Každý prvek pole reprezentuje jeden očekávaný vstup od studenta. Úloha tak může mít více polí (např. $D(f) = \dots$, *intervaly růstu*, *lokální extrém v $x = \dots$*).

Schema jednoho prvku:

```
{
  "key": "<identifikátor>",          # interní, pro logging
  "label_latex": "<LaTeX>",          # prefix před vstupem, např. "D(f) = "
  "type": "<typ>",                   # decimal | mathlive | multiple_choice |
    open_text
  "expected": <hodnota>,             # podle typu (viz níže)
  "tolerance": <float>,              # pouze pro type=decimal
  "options": [...],                 # pouze pro type=multiple_choice
}
```

3.4. Typy výsledků

Typ	expected	Vyhodnocení
decimal	float	MathLive vstup parsovaný přes Compute Engine <code>.N().valueOf()</code> ; porovnání numericky s <code>tolerance</code> .
mathlive	string (LaTeX)	Compute Engine 3-fázová strategie: <code>canonical.isSame() → simplify().isSame() → .N()</code> numericky.
multiple_choice	klíč správné možnosti	Radio buttony; <code>options[]</code> je pole <code>{key, label_latex}</code> .
open_text	vzorové řešení (string)	Žádné auto-vyhodnocení; doporučuje LLM (Gemini) nebo expertní revizi.

4. Vektor znalostí a kategorie

4.1. Dvě úrovně anotace úlohy

U každé úlohy rozlišujeme **dvě** ortogonální věci:

1. **Vektor znalostí** (`knowledge_vector`, 61 vah) — celý mix vlastností, typu, dovedností i kategorií SŠ/VŠ. Slovník přiřazující každé skill-komponentě procentuální váhu (0–100). Při řešení úlohy se přírůstek (úspěch) nebo úbytek (nesplnění) studentovy znalosti rozdělí mezi komponenty v poměru těchto vah. Paralelní vektor existuje i ve studentově profilu — tam ale hodnoty $\in [0, 1]$ (IRT/BKT).
2. **Anotace úlohy** (uložená „bokem“ pro pozdější automatické předvyplnění `knowledge_vector`):
 - `category` — jedna primární kategorie ze 45 (jen SŠ + VŠ matematika).
 - `properties` — multi-select 9 vlastností.
 - `task_type` — multi-select typů (zatím 1).
 - `skills` — multi-select 6 dovedností.

Anotace nemá žádný runtime efekt v IRT/BKT — slouží jen k tomu, aby z ní engine v další fázi vygeneroval výchozí distribuci vah v `knowledge_vectoru`, kterou expert ručně doladí.

4.2. KNOWLEDGE_WEIGHTS (61 vah, verze 260601_v2)

Definováno v `tasks/knowledge_weights.py`. Seznam byl revidován dr. Medkovou (verze 260601_v2) a obsahuje pouze *listové* váhy — hlavní (header) kategorie z původního excelu nejsou v seznamu. Všechny váhy jsou seskupené do 13 *skupin* pro účely barevného rozlišení v UI a budoucí paušální distribuce.

Skupiny vah (13). Každá váha nese prefix tvaru *Skupina - Název* (např. "Limita - LVL (krácení)"). Položky bez prefixu (*Konvexnost/konkávnost*, *Průběh funkce*) mají vlastní samostatnou skupinu.

Skupina	#	Barva v UI (CSS hex)
Vlastnosti	9	zelená #2e7d32
Typ	1	tmavě oranžová #ef6c00
Dovednosti	6	červenooranžová #d84315
SŠ	5	olivově žlutá #8d6e00
Funkce	7	modrá #1565c0
Monotonie	2	fialová #6a1b9a
Konvexnost/konkávnost	1	břidlicová #455a64
Spojitosť	2	tyrkysová #00695c
Limita	7	růžová #c2185b
Derivace	7	tmavě modrá #283593
Průběh funkce	1	uhlově šedá #424242
Primitivní funkce	5	tmavě červená #b71c1c
Určitý integrál	8	hnědá #4e342e
Celkem	61	

Barvy odpovídají třídám `.grp-*` v `templates/task_checker.html` a `templates/tasks_list.html`. Stejně barvy se používají třikrát:

1. v **textu labelu** každého slideru ve „Vektoru znalostí“v editoru,
2. v **textu** `<option>` v select boxu *Kategorie úlohy*,
3. v **rámu** `cat-chip` ve sloupci *Kategorie* v seznamu úloh.

4.3. TASK_CATEGORIES, TASK_PROPERTIES, TASK_TYPES, TASK_SKILLS

KNOWLEDGE_WEIGHTS (61) je v `tasks/knowledge_weights.py` rozdělen do čtyř disjunktních anotovaných podmnožin:

- **TASK_CATEGORIES** (45) — SŠ + Funkce + Monotonie + Konvexnost + Spojitost + Limita + Derivace + Průběh funkce + Primitivní funkce + Určitý integrál. Použito jako *single-select* v UI: jedna primární kategorie úlohy.
- **TASK_PROPERTIES** (9) — skupina *Vlastnosti*. *Multi-select* (checkboxy).
- **TASK_TYPES** (1) — skupina *Typ* (zatím jen *Aplikační*). *Multi-select*.
- **TASK_SKILLS** (6) — skupina *Dovednosti*. *Multi-select*.

Součet $45 + 9 + 1 + 6 = 61$ pokrývá celý KNOWLEDGE_WEIGHTS (sanity-check při importu modulu).

V další fázi (zatím **není implementováno**) bude každá kategorie + kombinace tagů mít defaultní distribuci vah, která se automaticky aplikuje na `knowledge_vector` při anotaci úlohy, a expert ji pak doladí ručně.

4.4. Kompletní seznam vah

Plný seznam 61 vah ve verzi 260601_v2, seskupený dle prefixu. Pořadí v tabulce odpovídá pořadí ve zdrojovém kódu i v UI.

#	Název	Stručná charakteristika
<i>Vlastnosti (9)</i>		
1	Vlastnosti - Lineární	Manipulace s lineárními výrazy a funkcemi $ax + b$.
2	Vlastnosti - Kvadratická	Kvadratické výrazy $ax^2 + bx + c$, kořeny, vrchol.
3	Vlastnosti - Mocninná	Mocninné funkce x^n , pravidla pro mocniny.
4	Vlastnosti - Logaritmická	Logaritmy a jejich vlastnosti.
5	Vlastnosti - Exponenciální	Exponenciální funkce a^x, e^x , vzorce.
6	Vlastnosti - Goniometrická	$\sin, \cos, \tan, \cot$ a jejich vzorce.
7	Vlastnosti - Absolutní hodnota	$ x $, rozklad podle znaménka.
8	Vlastnosti - Lomená lineární	Funkce $\frac{ax+b}{cx+d}$.
9	Vlastnosti - Zlomky	Operace se zlomky, společný jmenovatel, krácení.
<i>Typ (1)</i>		
10	Typ - Aplikační	Slovní / aplikační úlohy, modelace reálné situace.
<i>Dovednosti (6)</i>		
11	Dovednosti - Aplikace vzorce	Klasické vzorce $(a \pm b)^2, a^2 - b^2$, kosinová věta atd.
12	Dovednosti - Vytýkání, krácení	Algebraické zjednodušování, faktorizace.
13	Dovednosti - Roznásobení závorky	Distributivní zákon $a(b + c) = ab + ac$.
14	Dovednosti - Výpočet rovnic	Manipulace s rovnicemi (převody, substituce).
15	Dovednosti - Výpočet nerovnic	Řešení nerovnic, znaménková analýza.
16	Dovednosti - Derivování	Aplikace derivačních pravidel jako mechanická dovednost.
<i>SŠ (5)</i>		
17	SŠ - Algebraické výrazy	Úpravy, zjednodušování polynomiálních / racionálních výrazů.
18	SŠ - Elementární funkce	Vlastnosti základních funkcí bez derivace.
19	SŠ - Rovnice	Řešení rovnic vč. exp., log., goniometrických.
20	SŠ - Nerovnice	Řešení nerovnic s jednou neznámou.

#	Název	Stručná charakteristika
21	SŠ - Soustavy rovnic	Lineární soustavy 2–3 neznámých (dosazovací / sčítací).
<i>Funkce (7)</i>		
22	Funkce - Definiční obor	Stanovení D_f (odmocniny, jmenovatele, log, arcsin).
23	Funkce - Aritmetika	Operace $f + g, f \cdot g, f/g$ a jejich def. obory.
24	Funkce - Skládání/rozkládání	$f \circ g$, určení vnitřní / vnější funkce, iterace.
25	Funkce - Sudá/lichá	Symetrie funkce vzhledem k ose y resp. k počátku.
26	Funkce - Inverzní funkce	Stanovení f^{-1} , def. obor a obor hodnot.
27	Funkce - Tečna ke grafu	Rovnice tečny / normály v daném bodě.
28	Funkce - Asymptota	Vertikální, horizontální a šikmé asymptoty.
<i>Monotonie (2)</i>		
29	Monotonie - Absolutní extrémů na intervalu	Globální max/min na uzavřeném intervalu.
30	Monotonie - Určování lokálních extrémů	Lokální max/min z první (a druhé) derivace.
<i>Konvexnost/konkávnost (1)</i>		
31	Konvexnost/konkávnost	Intervaly konvexity, konkávnosti z f'' , inflexní body.
<i>Spojitosť (2)</i>		
32	Spojitosť - Spojitosť	Spojitosť v bodě, na intervalu; dodefinování.
33	Spojitosť - Bolzanova věta	Existence kořene v uzavřeném intervalu.
<i>Limita (7)</i>		
34	Limita - VOAL (dosazení)	Věta o aritmetice limit — dosazení do spojitě funkce.
35	Limita - LVL (krácení)	Typ $0/0$ řešený krácením společného činitele.
36	Limita - S odmocninou	Typ $0/0$ řešený rozšířením odmocninou.
37	Limita - Vytknutí nejvyšší mocniny	Typ ∞/∞ v $\pm\infty$.
38	Limita - Jednostranné limity	$\lim_{x \rightarrow a^\pm}$, neexistence dvoustranné limity.
39	Limita - Typová limita	Tabulkové limity ($\sin x/x$, $(e^x - 1)/x$, ...).
40	Limita - Lhopitalovo pravidlo	L'Hospitalovo pravidlo pro $0/0$ a ∞/∞ .
<i>Derivace (7)</i>		
41	Derivace - Sčítání	Pravidlo $(f + g)' = f' + g'$.
42	Derivace - Součin	Pravidlo $(fg)' = f'g + fg'$.
43	Derivace - Podíl	Pravidlo $(f/g)' = (f'g - fg')/g^2$.
44	Derivace - Složená funkce	Řetízkové pravidlo $(f \circ g)' = f'(g) \cdot g'$.
45	Derivace - Diferenciál	$df(x, h) = f'(x) \cdot h$; aproximace, odhady chyb.
46	Derivace - Taylorův polynom	Taylorův / Maclaurinův polynom řádu n .
47	Derivace - Vyšší řády	$f^{(n)}$ a jejich výpočet.
<i>Průběh funkce (1)</i>		
48	Průběh funkce	Komplexní vyšetření průběhu (D, parita, monotonie, ...).
<i>Primitivní funkce (5)</i>		
49	Primitivní funkce - Sčítání	Linearita integrálu.
50	Primitivní funkce - Per partes	$\int u dv = uv - \int v du$ pro neurčitý integrál.
51	Primitivní funkce - 1.věta o substituci	$\int f(g(x))g'(x) dx = \int f(t) dt$.
52	Primitivní funkce - 2.věta o substituci	Substituce $x = \varphi(t)$ (typicky s odmocninou nebo exp.).

#	Název	Stručná charakteristika
53	Primitivní funkce - Parciální zlomky	Rozklad racionální funkce, integrace.
<i>Určitý integrál (8)</i>		
54	Určitý integrál - Sčítání	Linearita určitého integrálu.
55	Určitý integrál - Aditivita	$\int_a^c f = \int_a^b f + \int_b^c f$ pro $a < b < c$.
56	Určitý integrál - Per partes	Per partes pro určitý integrál (s mezemi).
57	Určitý integrál - 1.věta o substituci	První věta s úpravou mezí.
58	Určitý integrál - 2.věta o substituci	Druhá věta s úpravou mezí.
59	Určitý integrál - Parciální zlomky	Parciální zlomky v určitém integrálu.
60	Určitý integrál - Nevlastní	Integrály s nekonečnými mezemi nebo neomezenou funkcí.
61	Určitý integrál - Obsah plochy	Výpočet obsahu rovinné oblasti ohraničené grafy funkcí.

Pozn. k drobným typografickým odchylkám. V seznamu se opakovaně objevuje „Vlastnosti“(1× „Vlastnosti“pro Goniometrickou) a „substituci“místo „substituci“— ponecháno přesně dle excelu dr. Medkové, ať se interní názvy nerozcházejí mezi zdroji.

5. Webové UI: Task Checker

5.1. Routes

Metoda	Cesta	Účel
GET / POST	/login	Formulář pro zadání hesla (session-based ochrana).
GET / POST	/logout	Smaže session, redirect na /login.
GET	/	Redirect na první úlohu (vyžaduje login).
GET	/tasks	Tabulka všech úloh (filtr podle kategorie, fulltext).
GET	/tasks/<id>	Detail úlohy — editor + sandbox.
POST	/api/tasks/<id>	Uložit změny (JSON body s editable poli).
DELETE	/api/tasks/<id>	Smazat úlohu.

5.2. Přihlášení a session

Všechny views kromě /login jsou chráněné dekorátorem `@require_login`. Při neautentizovaném přístupu se uživatel přesměruje na /login s parametrem `next` obsahujícím plnou URL původního požadavku (včetně reverse-proxy prefixu /adaptmath/). Po zadání správného hesla se vytvoří podepsaná session cookie a uživatel se vrátí na původní stránku.

Konfigurace přes env vars (systemd unit):

- APP_PASSWORD — heslo pro celou aplikaci (jeden globální účet pro tým),
- FLASK_SECRET_KEY — 64-znakový hex pro podepisování cookies,
- APP_SESSION_DAYS — platnost session v dnech (default 30).

Login screen má orange-accent design (logo „A“, brand *AdaptMath*, autofocus na input). V editoru je v hlavičce stránky odkaz *Odhlásit*.

5.3. Seznam úloh (/tasks)

Tabulka 415 úloh se sloupci: *Task ID*, *Náhled zadání* (KaTeX), *Typy výsledků* (badges), *Kategorie*, *Kognitivní zátěž*.

Sloupec Kategorie. Hodnota `t.category` z DB se zobrazí jako *cat-chip* (zaoblený rámeček) v barvě CSS třídy `.grp-XYZ` odpovídající skupině váhy. Pokud úloha kategorii ještě nemá nastavenou (typický stav po seedu), zobrazuje se světle šedý šrafovaný chip „— nepřirazen —“.

Filtr. Dropdown nad tabulkou nabízí všech 61 kategorií (text option barevně podle skupiny) plus speciální položku „— bez kategorie —“ pro rychlou identifikaci úloh, které tým ještě nezpracoval. Fulltext input hledá v `task_id`, hodnotě kategorie i v `content_latex`.

5.4. Editor úlohy (/tasks/<id>)

UI je rozděleno do dvou sloupců.

Levý sloupec (vizualizace + test).

- Živý KaTeX náhled zadání (re-renderuje při každém keystroke).
- Sandbox „Vyzkoušej řešení“ — MathLive vstupní pole; tlačítko *Otestovat* spustí vyhodnocení proti aktuálnímu (i neuloženému) **expected**.
- Sekce „Vzorové odpovědi“ — vyrenderované očekávané hodnoty.

Pravý sloupec (editor).

- Task ID, kognitivní zátěž (select A–F).
- *Kategorie úlohy* — single-select 45 položek (jen SŠ + VŠ matematika), text barevně rozlišený podle příslušné skupiny.
- Pod selektem 3 *multi-select* sekce s checkboxy — jsou barevně sladěné se skupinou:
 - *Vlastnosti* (9 položek, zelená),
 - *Typ* (zatím 1 položka — Aplikační, krémová),
 - *Dovednosti* (6 položek, oranžová).

Hodnoty se ukládají do JSON sloupců `properties`, `task_type`, `skills`.

- Zadání v LaTeXu (textarea, full-width).
- Editor výsledků — karta pro každý prvek pole `results`; lze přidat/odebrat; pro každý se mění `key`, `label_latex`, typ (select), a `expected` (specifické pro typ — MathLive pole, číselný input, MC options, textarea).

Pod editorem (full-width) — Vektor znalostí.

- Plochý seznam **61 vah** ve 2 sloupcích; každý řádek má label v barvě skupiny, range slider (0–100, krok 1), spojený number input a znak %.
- Nad gridem je **legenda** 13 skupin — každá s barevnou tečkou a jménem skupiny.
- Filtr textový („lim“, „derivace“) + checkbox *jen nenulové*.
- Sumarizace v hlavičce sekce: aktuální součet všech vah, počet nenulových z 61, tlačítko *Vynulovat vše*.
- Slider ↔ number input jsou dvousměrně synchronizované; aktivní řádky (váha > 0) jsou zvýrazněné a uloženy jako jediné v JSON (chybějící klíč = 0).

Sticky save bar (dole).

- *Uložit změny* — POST `/api/tasks/<id>` s celým stavem;
- *Vrátit* — obnoví poslední uložený stav;
- *Smazat úlohu* — DELETE endpoint.

Klávesové zkratky. $\leftarrow$ a $\rightarrow$ pro navigaci mezi úlohami, `Ctrl+S` (resp. `Cmd+S`) pro uložení.

5.5. Tipy pro vyhodnocení MathLive Compute Engine

- `mathlive` odpovědi se ukládají jako čistý LaTeX. Compute Engine je tolerantní k variantám zápisu zlomku, znamének, `\infty` vs `+\infty`, atd.
- Pro intervaly preferujeme standardní zápis: `(a, b)` otevřený, `[a, b]` uzavřený, `\cup` pro sjednocení.
- Tam, kde by Compute Engine selhal (cyklické sjednocení po $k \in \mathbb{Z}$, „limita neexistuje“ jako popis), používáme `multiple_choice` se třemi variantami (1 správná + 2 typické chyby).

6. Datový obsah

6.1. Zdroj úloh

Skriptu *Základy matematiky 1* (P. Pražák, P. Bauer; KIKM FIM UHK). Zdrojové `.tex` soubory cvičení 1–13 byly ručně (LLM-asistovaně) interpretovány a převedeny do strukturovaného formátu v `tasks/cvNN.py`.

6.2. Pokrytí

Cvičení	Téma	Úloh
cv01	Funkce, definiční obor, kvadratická funkce	37
cv02	Skládání, prostá, inverzní funkce	29
cv03	Monotonie, parita, polynomy, Hornerovo schéma	42
cv04	Limita — aritmetika limit	18
cv05	Limita — věta o složené funkci, typové limity	44
cv06	Spojitost funkce	23
cv07	Derivace (součet, součin, podíl, řetězkové pravidlo)	47
cv08	Tečna, normála, absolutní extrémy, L'Hospital	36
cv09	1. derivace, lokální extrémy, optimalizace	26
cv10	2. derivace (konvexita, inflexe), asymptoty	18
cv11	Diferenciál, Taylorův a Maclaurinův polynom	27
cv12	Primitivní funkce (linearita, per-partes, substituce)	58
cv13	Integrace racionálních funkcí	10
Celkem		415

6.3. Distribuce typů výsledků

Typ	Počet
<code>mathlive</code>	319
<code>multiple_choice</code>	115
<code>decimal</code>	83
<code>open_text</code>	0 [*]

^{*}Původní negace výroků byly převedeny na `multiple_choice` se 3 možnostmi (správná + 2 typické chyby studentů).

7. Spuštění lokálně

7.1. Závislosti

- Python 3.10+
- Docker (pro PostgreSQL) — nebo systémový PostgreSQL 14+
- Python balíčky: flask, sqlalchemy, psycopg2-binary

7.2. První spuštění

```
# 1) klon repozitare
git clone https://github.com/dominikpalla/adapt_math.git
cd adapt_math

# 2) Python venv + dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install flask sqlalchemy psycopg2-binary

# 3) Spustit PostgreSQL kontejner (lokálně)
docker compose up -d

# 4) Naplnit DB (drop & recreate + insert 415 uloh)
python seed_db.py

# 5) Spustit Flask
python app.py
```

Aplikace běží na <http://127.0.0.1:5000>. Přihlašovací heslo je default kirkmjenejlepsi (env APP_PASSWORD).

7.3. Konfigurace přes env vars

Pro produkci preferujeme všechny citlivé hodnoty mimo zdrojový kód:

Env var	Účel
DATABASE_URL	Připojení k PostgreSQL (postgresql://...).
APP_PASSWORD	Heslo pro Flask login screen.
FLASK_SECRET_KEY	Hex string pro podepisování session cookies.
APP_SESSION_DAYS	Doba platnosti session (default 30).

7.4. Opětovné nasazení po změně schématu

Jelikož v aktuální fázi nepoužíváme migrace (Alembic), po každé změně sloupců v model.py stačí spustit znovu `python seed_db.py` — skript provede `DROP TABLE ... CASCADE` a `CREATE TABLE` dle aktuálního ORM modelu, načež úlohy vloží znovu.

Pozor: drop & recreate ztrácí veškerá data nasazená přes UI (ručně upravené kategorie, vektory znalostí). Pro produkci je nutné zavést migrace.

8. Nasazení do produkce (UHK)

Aplikace je nasazena na školní VM na adrese:

<https://moodlefim.uhk.cz/adaptmath/>

Server je sdílený s existující Moodle instancí (FIM UHK, vnitřní IP 172.25.14.164, Apache 2.4 + MariaDB). AdaptMath běží *vedle* Moodlu jako Apache reverse proxy do lokální Flask aplikace, bez jakéhokoli zásahu do běhu Moodlu.

8.1. Architektura nasazení

1. **Flask aplikace** běží jako systemd služba `adaptmath.service` pod uživatelem `moodlepalla` a naslouchá pouze na `127.0.0.1:5000` (loopback, ven se nedostane).
2. **PostgreSQL 14** (systémový, ne Docker) běží na `127.0.0.1:5432` s databází `adaptmath` a uživatelem `adaptmath_user`.
3. **Apache vhost** `moodle-ssl.conf` dělá reverse proxy:

```
RedirectMatch permanent ^/adaptmath$ /adaptmath/
ProxyPreserveHost On
ProxyPass          /adaptmath/ http://127.0.0.1:5000/
ProxyPassReverse   /adaptmath/ http://127.0.0.1:5000/
RequestHeader set  X-Forwarded-Prefix "/adaptmath"
```

4. Flask aplikace používá `werkzeug.middleware.proxy_fix.ProxyFix`, který čte `X-Forwarded-Prefix` z hlavičky a automaticky prependuje prefix k výstupům `url_for()`, takže všechny linky v UI vedou na `/adaptmath/...`

8.2. Systemd unit

Soubor `/etc/systemd/system/adaptmath.service` (chmod 600 kvůli `FLASK_SECRET_KEY`):

```
[Unit]
Description=AdaptMath Task Checker (Flask)
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=simple
User=moodlepalla
WorkingDirectory=/home/moodlepalla/adapt_math
Environment="DATABASE_URL=postgresql://adaptmath_user:***@localhost:5432/adaptmath"
Environment="APP_PASSWORD=***"
Environment="FLASK_SECRET_KEY=<64 hex chars>"
Environment="APP_SESSION_DAYS=30"
ExecStart=/home/moodlepalla/adapt_math/.venv/bin/python -m flask --app app
        run --host 127.0.0.1 --port 5000
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

8.3. Provozní příkazy

Akce	Příkaz
Status	<code>sudo systemctl status adaptmath</code>
Logy (live)	<code>sudo journalctl -u adaptmath -f</code>
Restart	<code>sudo systemctl restart adaptmath</code>
Update kódu	<code>cd ~/adapt_math && git pull && sudo systemctl restart adaptmath</code>
Změnit heslo	<code>edit adaptmath.service</code> , pak <code>daemon-reload + restart</code>
Reset DB	<code>DATABASE_URL=... .venv/bin/python seed_db.py</code>
Vrátit vhost	<code>sudo cp moodle-ssl.conf.bak.* moodle-ssl.conf + reload apache2</code>

8.4. Známá omezení

- **Werkzeug dev server** — aktuálně Flask běží přes `flask run`, což je vývojový server. Pro skutečnou produkci by bylo lepší přepnout na `gunicorn` (jednoduchá změna v `ExecStart`).
- **SSL certifikát** — ručně instalovaný v `/etc/ssl/certs/moodlefim.crt`, vyžaduje pravidelnou obnovu (řeší IT UHK; není Let's Encrypt s auto-renew).
- **Žádné DB migrace** — viz „Opětovné nasazení po změně schématu“ výše.
- **Jediný globální účet** — všichni členové týmu sdílí jedno heslo. Pro plnohodnotný multi-user režim budeme potřebovat Flask-Login a tabulku `users`.

9. Co dál (roadmap)

9.1. Krátkodobé

1. **Tagování úloh experty.** Tým musí manuálně projít 415 úloh a přiřadit `category` + nastavit `knowledge_vector`. To je hlavní úkol Task Checkeru.
2. **Distribuce vah dle kategorie.** Definovat slovník `CATEGORY_DEFAULT_VECTOR`, který pro každou ze 61 kategorií předvyplní výchozí distribuci 61 vah. Při výběru kategorie v UI se předvyplní `knowledge_vector`, expert pak ladí.
3. **LLM vyhodnocení open_text.** Aktuálně máme všechny negace převedeny na `multiple_choice`; pro budoucí volné úlohy zavést endpoint `POST /api/eval-open-text` volající Gemini.
4. **Alembic migrace.** Nahradit `drop & recreate` řízenými migracemi.
5. **Gunicorn pro produkci.** Nahradit Werkzeug dev server na produkci.

9.2. Střednědobé

1. **IRT engine (py-irt).** Pilotní tréninková fáze pro kalibraci obtížnosti a diskriminace. Vlastní route `/student/<id>/next-task`, která vybere úlohu podle aktuálního θ studenta.
2. **BKT engine (py-bkt).** Po každé interakci aktualizuje studentův vektor znalostí na základě `knowledge_vector` úlohy a parametrů α, β .
3. **Studentský profil.** Tabulka `students` s preferencemi (styl učení, motivace, math anxiety) a vektorem znalostí (61 hodnot v $[0, 1]$).
4. **Multi-user režim.** Tabulka `users` + Flask-Login pro plnohodnotné přihlašování více účtů (zatím sdílíme jedno heslo).
5. **Logování interakcí** pro výzkum (čas řešení, jistota, AI nápověda).

9.3. Dlouhodobé

1. **Integrace s Moodle.** Engine jako externí Moodle plugin / LTI tool. Výstup TQ23000206-V3.

2. **Generování úloh LLM.** Tvorba variant na téma „kategorie + obtížnost“.
3. **Dashboard pro experty.** Vizualizace pokroku jednotlivých studentů, analýza odchylek od průměru.

10. Reference

- Repozitář projektu: https://github.com/dominikpalla/adapt_math
- Skripta *Základy matematiky 1*, P. Pražák, P. Bauer, FIM UHK
- MathLive: <https://github.com/arnog/mathlive>
- Compute Engine: <https://cortexjs.io/compute-engine/>
- KaTeX: <https://katex.org/>
- py-irt: <https://github.com/nd-ball/py-irt>
- py-bkt: <https://github.com/CAHLR/pyBKT>

Tato dokumentace byla vygenerována ve fázi plnění databáze úloh (Task Checker, verze 1.0). Bude rozšířena po implementaci IRT/BKT engine a integrace s Moodle.